EX 4 : IMPLEMENTATION OF A CONVOLUTIONAL NEURAL NETWORK (CNN)

AIM:

To Implement a Convolutional Neural Network (CNN) from scratch using PyTorch to classify images. Train the network using a dataset of labeled images and evaluate its performance. Additionally, visualize the learned filters in the convolution layers.

OBJECTIVES:

1. Understand the architecture and functionality of Convolutional Neural Networks (CNNs).
2. Implement CNN layers including convolution, pooling, and fully connected layers.
3. Train the model on the CIFAR-10 dataset and evaluate its performance.
4. Visualize the learned filters to interpret feature extraction at early layers.

ALGORITHM:

1. Import the required libraries, including PyTorch, torchvision, and Matplotlib.
2. Load the CIFAR-10 dataset for training and testing.
3. Convert images into tensors and normalize pixel values.
4. Create DataLoaders with a batch size of 64 to process images in batches.
5. Define the CNN architecture with two convolutional layers, ReLU activation, max-pooling layers, and fully connected layers.
6. Initialize the CNN model and select the available device (GPU or CPU).
7. Define the cross-entropy loss function and Adam optimizer with a learning rate of 0.001.
8. Train the model for 10 epochs by performing forward propagation, calculating loss, backpropagation, and updating weights.
9. Display the average training loss after each epoch.
10. Evaluate the trained model using the CIFAR-10 test dataset and calculate classification accuracy.
11. Visualize the learned filters of the first convolutional layer using Matplotlib.
12. Analyze the test accuracy and visualized filters to understand how the CNN learns image features.

SOURCE CODE:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        (0.5, 0.5, 0.5),
        (0.5, 0.5, 0.5)
    )
])
trainset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=64,
    shuffle=True
)
testset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

testloader = torch.utils.data.DataLoader(
    testset,
    batch_size=64,
    shuffle=False
)
classes = trainset.classes

print("CIFAR-10 dataset loaded successfully!")
```

```
print("Number of training images:", len(trainset))
print("Number of testing images:", len(testset))
print("Classes:", classes)
class SimpleCNN(nn.Module):

    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(
            3, 16, 3, padding=1
        )
        self.pool = nn.MaxPool2d(
            2, 2
        )
        self.conv2 = nn.Conv2d(
            16, 32, 3, padding=1
        )
        self.fc1 = nn.Linear(
            32 * 8 * 8,
            128
        )

        self.fc2 = nn.Linear(
            128,
            10
        )
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(
            self.relu(
                self.conv1(x)
            )
        )
        x = self.pool(
            self.relu(
                self.conv2(x)
            )
        )
        x = x.view(
            -1,
            32 * 8 * 8
```

```
        )
        x = self.relu(
            self.fc1(x)
        )
        x = self.fc2(x)

        return x
device = torch.device(
    "cuda" if torch.cuda.is_available()
    else "cpu"
)

print("\nUsing device:", device)

# Create model
model = SimpleCNN().to(device)

# Loss function
lossfn = nn.CrossEntropyLoss()

# Adam optimizer
optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)

# Training
epochs = 10

for epoch in range(epochs):
    running_loss = 0.0
    model.train()
    for inputs, labels in trainloader:
        inputs = inputs.to(device)
        labels = labels.to(device)
        # Clear gradients
        optimizer.zero_grad()
        # Forward pass
        outputs = model(inputs)
        # Calculate loss
        loss = lossfn(
```

```
            outputs,
            labels
        )
        # Backward pass
        loss.backward()
        # Update weights
        optimizer.step()
        running_loss += loss.item()
    avg_loss = running_loss / len(trainloader)
    print(
        f"Epoch {epoch + 1}, "
        f"Loss: {avg_loss:.4f}"
    )



#
  ============================================================
  ===
# Step 4: Evaluate Model Performance
#
  ============================================================
  ===

correct = 0
total = 0

model.eval()

with torch.no_grad():

    for images, labels in testloader:

        images = images.to(device)
        labels = labels.to(device)

        # Prediction
        outputs = model(images)

        _, predicted = outputs.max(1)

        total += labels.size(0)
```

```
        correct += (
            predicted == labels
        ).sum().item()

accuracy = 100 * correct / total

print(
    f"\nAccuracy on test data: "
    f"{accuracy:.2f}%"
)



#
  ============================================================
  ===
# Step 5: Visualize Learned Filters
#
  ============================================================
  ===

def visualize_filters(
    layer,
    n_filters=8
):

    # Get filters from convolutional layer
    filters = (
        layer.weight.data
        .clone()
        .cpu()
    )

    fig, axs = plt.subplots(
        1,
        n_filters,
        figsize=(15, 4)
    )

    for i in range(n_filters):

        f = filters[i]
```

```
        # Normalize filter values
        f = (
            f - f.min()
        ) / (
            f.max() - f.min()
        )

        # Display filter
        axs[i].imshow(
            f.permute(1, 2, 0)
        )

        axs[i].axis('off')

        axs[i].set_title(
            f'Filter {i + 1}'
        )

    plt.tight_layout()
    plt.show()

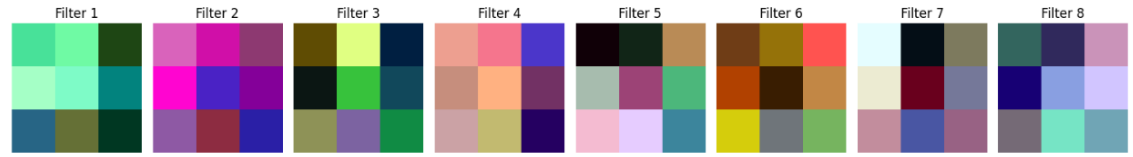

# Visualize first convolutional layer filters
visualize_filters(
    model.conv1,
    n_filters=8
)
```

OUTPUT:

```
100%|████████| 170M/170M [26:32<00:00, 107kB/s]
CIFAR-10 dataset loaded successfully!
Number of training images: 50000
Number of testing images: 10000
Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

Using device: cpu
Epoch 1, Loss: 1.4522
Epoch 2, Loss: 1.1238
Epoch 3, Loss: 0.9763
Epoch 4, Loss: 0.8765
Epoch 5, Loss: 0.8002
Epoch 6, Loss: 0.7311
Epoch 7, Loss: 0.6683
Epoch 8, Loss: 0.6135
Epoch 9, Loss: 0.5561
Epoch 10, Loss: 0.5048

Accuracy on test data: 68.99%
```



+ Code + Text

RESULT:

This experiment successfully demonstrates how CNNs extract hierarchical spatial features for image classification. By training on CIFAR-10, we observed performance improvements and explored how early layers learn to detect basic patterns using visualized filters. This forms the basis for more advanced deep learning models used in computer vision.